

LAB TASKS

CSC241 - Object-Oriented Programming | Lab 02

Classes, Objects, Data Members & Member Functions

Student Name	Sharmeen Imtiaz
Roll No.	SP26-BAI-045
Program	BS Artificial Intelligence (OOP - Object Oriented Programming)
Course	CSC241 - Object-Oriented Programming
Lab	Lab 02 - Classes, Objects, Data Members & Member Functions
Section	BAI
Semester	Fall 2026
Instructor	Muhammad Shahid Bhatti

Each task below shows the Java source code followed by a Command Prompt screenshot of that program compiling and running.

Task 1 - Build a Class and Prove Independent Object State

Concept focus: Class anatomy, object instantiation, reference variables, identity/state, instance variables.

Code: Student.java

```
// Student.java
// CSC241 - Lab 02 - Task 1 & Task 2
// Sharmeen Imtiaz | SP26-BAI-045

class Student {
    // Instance variables (Task 1)
    String studentId;
    String name;
    int completedCredits;

    // Task 2: Add credits to this student's record
    void addCredits(int credits) {
        completedCredits = completedCredits + credits;
    }

    // Task 2: Compute how many credits remain to reach a degree total
    int remainingCredits(int degreeCredits) {
        int remaining = degreeCredits - completedCredits;
        return remaining;
    }

    // Task 2: Build a one-line readable summary of this student
    String summary() {
        return "ID: " + studentId + " | Name: " + name + " | Completed Credits: " +
        completedCredits;
    }
}
```

Code: Task1Demo.java

```
// Task1Demo.java
// CSC241 - Lab 02 - Task 1: Build a Class and Prove Independent Object State
// Sharmeen Imtiaz | SP26-BAI-045

class Task1Demo {
    public static void main(String[] args) {

        // Create three Student objects from one Student class
        Student s1 = new Student();
        Student s2 = new Student();
        Student s3 = new Student();

        // Assign values (BAI example data)
        s1.studentId = "SP26-BAI-001";
        s1.name = "Abeer Amina";
    }
}
```

```

        s1.completedCredits = 30;

        s2.studentId = "SP26-BAI-002";
        s2.name = "Ali Ishtiaq";
        s2.completedCredits = 45;

        s3.studentId = "SP26-BAI-003";
        s3.name = "Abdul Rehman Azam";
        s3.completedCredits = 60;

        // Print all three objects before any change
        System.out.println("Before change:");
        System.out.println(s1.studentId + " | " + s1.name + " | " +
s1.completedCredits);
        System.out.println(s2.studentId + " | " + s2.name + " | " +
s2.completedCredits);
        System.out.println(s3.studentId + " | " + s3.name + " | " +
s3.completedCredits);

        // Increase completedCredits of only one object (s2)
        s2.completedCredits = s2.completedCredits + 3;

        // Print all three objects again
        System.out.println("\nAfter change (only s2 modified):");
        System.out.println(s1.studentId + " | " + s1.name + " | " +
s1.completedCredits);
        System.out.println(s2.studentId + " | " + s2.name + " | " +
s2.completedCredits);
        System.out.println(s3.studentId + " | " + s3.name + " | " +
s3.completedCredits);

        // Explanation:
        // s1 and s3 are separate objects in separate memory locations, each with
        // its own copy of completedCredits, so changing s2's field cannot affect
        them.
    }
}

```

CMD Output: javac Student.java Task1Demo.java → java Task1Demo

```

Lab02 - Task 1 - Task1Demo
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>javac Student.java Task1Demo.java
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>java Task1Demo
Before change:
SP26-BAI-001 | Abeer Amina | 30
SP26-BAI-002 | Ali Ishtiaq | 45
SP26-BAI-003 | Abdul Rehman Azam | 60

After change (only s2 modified):
SP26-BAI-001 | Abeer Amina | 30
SP26-BAI-002 | Ali Ishtiaq | 48
SP26-BAI-003 | Abdul Rehman Azam | 60
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>

```

Task 1 Concept Check

1. What is the difference between the Student class and the objects s1, s2, and s3 created from it?

Answer: Student is the blueprint that defines what fields an object of this type will have, while s1, s2, and s3 are actual objects created from that blueprint, each occupying its own memory and holding its own values.

2. Why does changing s2.completedCredits not change the completedCredits value stored in s1 or s3?

Answer: Each object stores its own independent copy of the instance variables in a separate memory location, so modifying one object's field has no effect on any other object's field.

3. Which variables in Student represent persistent object state, and why are they instance variables rather than local variables?

Answer: studentId, name, and completedCredits are instance variables because they must persist for the entire lifetime of each object and differ from object to object; local variables would disappear as soon as a method finished executing.

Task 2 - Add Behavior - Parameters, Local Variables, and Return Values

Concept focus: Instance methods, receiver object, parameters vs. arguments, local variables, void and non-void return types.

Code: Task2Demo.java (uses updated Student.java)

```
// Task2Demo.java
// CSC241 - Lab 02 - Task 2: Add Behavior - Parameters, Local Variables, Return
// Values
// Sharmeen Imtiaz | SP26-BAI-045

class Task2Demo {
    public static void main(String[] args) {

        Student s1 = new Student();
        s1.studentId = "SP26-BAI-001";
        s1.name = "Abeer Amina";
        s1.completedCredits = 30;

        Student s2 = new Student();
        s2.studentId = "SP26-BAI-002";
        s2.name = "Ali Ishtiaq";
        s2.completedCredits = 45;

        // Call addCredits on both objects
        s1.addCredits(6);
        s2.addCredits(9);

        // Call remainingCredits on both objects (degree requires 130 credits)
        int remaining1 = s1.remainingCredits(130);
        int remaining2 = s2.remainingCredits(130);

        System.out.println(s1.name + " remaining credits: " + remaining1);
        System.out.println(s2.name + " remaining credits: " + remaining2);

        // Call summary on both objects
        System.out.println(s1.summary());
        System.out.println(s2.summary());

        // credits (in addCredits) is a PARAMETER - a value passed in when the
        // method is called.
        // completedCredits is a FIELD - it belongs to the object and persists
        // across calls.
        // remaining (in remainingCredits) is a LOCAL VARIABLE - it exists only
        // inside that method call.
    }
}
```

CMD Output: `javac Student.java Task2Demo.java → java Task2Demo`

```
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>javac Student.java Task2Demo.java
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>java Task2Demo
Abeer Amina remaining credits: 94
Ali Ishtiaq remaining credits: 76
ID: SP26-BAI-001 | Name: Abeer Amina | Completed Credits: 36
ID: SP26-BAI-002 | Name: Ali Ishtiaq | Completed Credits: 54
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>
```

Task 2 Concept Check

1. In addCredits(int credits), what is the difference between the parameter credits and the object field completedCredits?

Answer: credits is a parameter - a temporary value passed in only for the duration of that one method call - while completedCredits is a field belonging to the object that keeps its value between calls.

2. Why is remaining inside remainingCredits(...) better as a local variable than as an instance field?

Answer: remaining is only needed while the method is computing its answer, so it should not outlive the call or be visible outside the method the way an instance field would be.

3. What information does the return type of remainingCredits(int degreeCredits) give to the caller?

Answer: The int return type tells the caller that the method will always hand back a whole number, which the caller can store in a variable, print, or use in further calculations.

Task 3 - Method Signatures and Overloading

Concept focus: Method signatures, parameter count/type/order, compile-time overload resolution, return type not part of the signature.

Code: OverloadDemo.java

```
// OverloadDemo.java
// CSC241 - Lab 02 - Task 3: Method Signatures and Overloading
// Sharmeen Imtiaz | SP26-BAI-045

class OverloadDemo {

    // Signature: enroll(String)
    void enroll(String courseCode) {
        System.out.println("Enrolled in course: " + courseCode);
    }

    // Signature: enroll(String, int)
    void enroll(String courseCode, int section) {
        System.out.println("Enrolled in course: " + courseCode + ", Section: " +
section);
    }

    // Signature: enroll(int)
    void enroll(int numericCourseCode) {
        System.out.println("Enrolled in numeric course code: " +
numericCourseCode);
    }

    public static void main(String[] args) {
        OverloadDemo demo = new OverloadDemo();

        // Predict before compiling:
        demo.enroll("CSC241");           // calls enroll(String)
        demo.enroll("CSC241", 2);        // calls enroll(String, int)
        demo.enroll(241);                // calls enroll(int)

        // Invalid calls (kept commented out - would not compile):
        // demo.enroll();                 // ERROR: no matching method - enroll()
does not exist
        // demo.enroll("241", "2");      // ERROR: no enroll(String, String)
overload exists

        // Invalid overload attempt (differs only by return type) - observed
compiler error:
        // int enroll(String courseCode) { return 0; }
        // Compiler error (captured in compiler_notes.txt):
        // "error: method enroll(String) is already defined in class OverloadDemo"
        // Return type is not part of a method's signature, so this is a duplicate
method,
        // not a valid overload - the compiler cannot tell them apart.
```

```
}  
}
```

CMD Output: `javac OverloadDemo.java` → `java OverloadDemo` (valid overloads)

Lab02 - Task 3 - OverloadDemo

```
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>javac OverloadDemo.java  
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>java OverloadDemo  
Enrolled in course: CSC241  
Enrolled in course: CSC241, Section: 2  
Enrolled in numeric course code: 241  
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>
```

CMD Output: Invalid overload differing only by return type (compiler_notes.txt evidence)

Lab02 - Task 3 - Invalid Overload (compiler_notes.txt evidence)

```
// Testing an invalid overload that differs only by return type:  
// int enroll(String courseCode) { return 0; }  
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>javac InvalidOverloadTest.java  
InvalidOverloadTest.java:6: error: method enroll(String) is already defined in class  
    InvalidOverloadTest  
        int enroll(String courseCode) {  
            ^  
1 error  
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>
```

Task 3 Concept Check

1. What is the method signature of `enroll(String courseCode, int section)`?

Answer: `enroll(String, int)`.

2. Why are `enroll(String)` and `enroll(int)` valid overloads, while two methods with the same name and parameter types but different return types are not?

Answer: `enroll(String)` and `enroll(int)` differ in parameter type, which the compiler can use to distinguish calls; two methods with identical parameter lists look the same to the compiler when matching a call, so return type alone cannot disambiguate them - this caused the 'already defined' error shown above.

3. For the call `enroll("CSC241", 2)`, which overload should the compiler select, and what information does it use to make that decision?

Answer: The compiler selects `enroll(String, int)` based on the number and types of the arguments supplied in the call, not on how the returned value is used.

Task 4 - Prove Java Pass-by-Value

Concept focus: Primitive values are copied; object reference values are also copied; object mutation differs from parameter reassignment.

Code: PassByValueDemo.java

```
// PassByValueDemo.java
// CSC241 - Lab 02 - Task 4: Prove Java Pass-by-Value
// Sharmeen Imtiaz | SP26-BAI-045

class PassByValueDemo {

    // Experiment A: primitive int
    void changeNumber(int x) {
        x = 99;
    }

    // Experiment B: mutate the object through a copied reference
    void changeStudent(Student st) {
        st.completedCredits = 99;
    }

    // Experiment C: reassign the parameter to a new object
    void replaceStudent(Student st) {
        st = new Student();
        st.name = "Temporary";
    }

    public static void main(String[] args) {
        PassByValueDemo demo = new PassByValueDemo();

        // ----- Experiment A -----
        int number = 10;
        System.out.println("Experiment A - Before: " + number);
        demo.changeNumber(number);
        System.out.println("Experiment A - After: " + number);
        // The argument value is copied into the parameter, so changing x
        // inside changeNumber does not affect the caller's original int variable.

        // ----- Experiment B -----
        Student st1 = new Student();
        st1.name = "Abeer Amina";
        st1.completedCredits = 30;
        System.out.println("\nExperiment B - Before: " + st1.completedCredits);
        demo.changeStudent(st1);
        System.out.println("Experiment B - Inside Method: completedCredits set to
99");
        System.out.println("Experiment B - After: " + st1.completedCredits);
        // The argument value is copied into the parameter, but that copied value
is a
        // reference pointing to the same object, so mutating a field through it is
```

visible to the caller.

```
// ----- Experiment C -----
Student st2 = new Student();
st2.name = "Ali Ishtiaq";
System.out.println("\nExperiment C - Before: " + st2.name);
demo.replaceStudent(st2);
System.out.println("Experiment C - Inside Method: local st reassigned to a
new Student named Temporary");
System.out.println("Experiment C - After: " + st2.name);
// The argument value is copied into the parameter, so reassigning the
local
// parameter st only redirects that copy - it cannot redirect the caller's
original variable.
    }
}
```

CMD Output: `javac PassByValueDemo.java → java PassByValueDemo`

Lab02 - Task 4 - PassByValueDemo

```
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>javac PassByValueDemo.java
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>java PassByValueDemo
Experiment A - Before: 10
Experiment A - After: 10

Experiment B - Before: 30
Experiment B - Inside Method: completedCredits set to 99
Experiment B - After: 99

Experiment C - Before: Ali Ishtiaq
Experiment C - Inside Method: local st reassigned to a new Student named Temporary
Experiment C - After: Ali Ishtiaq
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>
```

Task 4 Concept Check

1. After `changeNumber(int x)` assigns `x = 99`, why does the caller's original `int` variable keep its old value?

Answer: The argument value is copied into the parameter, so `changeNumber` only ever modifies its own local copy `x` and never touches the original variable back in main.

2. If a `Student` reference is passed to a method, why can changing `st.completedCredits` be visible to the caller?

Answer: The reference value copied into `st` still points to the same `Student` object as the caller's variable, so writing to `st.completedCredits` changes the one shared object that both variables refer to.

3. Why does assigning `st = new Student()` inside a method not make the caller's variable refer to the new `Student` object?

Answer: That assignment only changes what the local copy of the reference points to; the argument value is copied into the parameter, so the caller's original variable still holds its original reference, which was never touched.

Task 5 - Integrated BAI Challenge - AIExperiment

Scenario: Design a class that stores AI experiment state and provides methods to update and query that state, using only concepts covered through Lecture 4.

Code: *AIExperiment.java*

```
// AIExperiment.java
// CSC241 - Lab 02 - Task 5: Integrated BAI Challenge
// Sharmeen Imtiaz | SP26-BAI-045

class AIExperiment {
    // Instance variables (R1)
    String experimentName;
    int completedEpochs;
    int targetEpochs;

    // R3: increase completedEpochs
    void runEpochs(int epochs) {
        completedEpochs = completedEpochs + epochs;
    }

    // R4: overloaded runEpochs with bonus epochs
    void runEpochs(int epochs, int bonusEpochs) {
        int totalEpochs = epochs + bonusEpochs;
        completedEpochs = completedEpochs + totalEpochs;
    }

    // R5: remaining epochs to reach the target
    int remainingEpochs() {
        return targetEpochs - completedEpochs;
    }

    // R6: readable one-line summary
    String status() {
        return "Experiment: " + experimentName + " | Completed: " + completedEpochs
            + "/" + targetEpochs + " epochs | Remaining: " + remainingEpochs();
    }
}
```

Code: *AIExperimentDemo.java*

```
// AIExperimentDemo.java
// CSC241 - Lab 02 - Task 5: Integrated BAI Challenge
// Sharmeen Imtiaz | SP26-BAI-045

class AIExperimentDemo {

    // R9: helper method that mutates an AIExperiment object through a copied
    reference
    void boostEpochs(AIExperiment exp, int extra) {
```

```

        exp.completedEpochs = exp.completedEpochs + extra;
    }

    public static void main(String[] args) {
        AIExperimentDemo demo = new AIExperimentDemo();

        // R2: two independent AIExperiment objects
        AIExperiment exp1 = new AIExperiment();
        exp1.experimentName = "ImageClassifier";
        exp1.completedEpochs = 5;
        exp1.targetEpochs = 20;

        AIExperiment exp2 = new AIExperiment();
        exp2.experimentName = "TextSummarizer";
        exp2.completedEpochs = 8;
        exp2.targetEpochs = 25;

        System.out.println("Initial state:");
        System.out.println(exp1.status());
        System.out.println(exp2.status());

        // R3 & R4: call runEpochs (single arg) and runEpochs (overloaded, two
args)
        exp1.runEpochs(3);
        exp2.runEpochs(2, 1);

        System.out.println("\nAfter runEpochs calls:");
        System.out.println(exp1.status());
        System.out.println(exp2.status());

        // R8: demonstrate that changing one experiment does not change the other
        System.out.println("\nR8 check - exp1 and exp2 remain independent:");
        System.out.println("exp1 completedEpochs: " + exp1.completedEpochs);
        System.out.println("exp2 completedEpochs: " + exp2.completedEpochs);

        // R9: pass exp1 to a helper method that modifies completedEpochs
        demo.boostEpochs(exp1, 4);
        System.out.println("\nAfter boostEpochs(exp1, 4):");
        System.out.println(exp1.status());
        // The mutation is visible because the reference value is copied into the
        // parameter exp, and that copy still points to the same object as the
        caller's exp1.

        System.out.println("\nFinal remaining epochs:");
        System.out.println(exp1.experimentName + " remaining: " +
exp1.remainingEpochs());
        System.out.println(exp2.experimentName + " remaining: " +
exp2.remainingEpochs());
    }
}

```

CMD Output: `javac AIExperiment.java AIExperimentDemo.java` → `java AIExperimentDemo`

```
Lab02 - Task 5 - AIExperimentDemo
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>javac AIExperiment.java AIExperimentDemo.java
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>java AIExperimentDemo
Initial state:
Experiment: ImageClassifier | Completed: 5/20 epochs | Remaining: 15
Experiment: TextSummarizer | Completed: 8/25 epochs | Remaining: 17

After runEpochs calls:
Experiment: ImageClassifier | Completed: 8/20 epochs | Remaining: 12
Experiment: TextSummarizer | Completed: 11/25 epochs | Remaining: 14

R8 check - exp1 and exp2 remain independent:
exp1 completedEpochs: 8
exp2 completedEpochs: 11

After boostEpochs(exp1, 4):
Experiment: ImageClassifier | Completed: 12/20 epochs | Remaining: 8

Final remaining epochs:
ImageClassifier remaining: 8
TextSummarizer remaining: 14
C:\CSC241\CIIT-SP26-BAI-045-LHR\Lab02>
```

Task 5 Concept Check

1. In `AIExperiment`, which data should be stored as instance variables, and which calculations should remain local to methods?

Answer: `experimentName`, `completedEpochs`, and `targetEpochs` are instance variables because they represent the ongoing state of each experiment across many calls, while intermediate totals used inside a method (such as `totalEpochs` in the overloaded `runEpochs`) stay as local variables.

2. Identify one method in your `AIExperiment` class that changes object state and one method that only computes or returns information.

Answer: `runEpochs(...)` changes object state by increasing `completedEpochs`, while `remainingEpochs()` and `status()` only read the existing fields and return information without modifying anything.

3. If you created overloaded methods in this task, state their signatures and explain why the compiler can distinguish them.

Answer: The overloaded methods are `runEpochs(int)` and `runEpochs(int, int)`; the compiler can tell them apart because they have a different number of parameters, even though both share the same method name.

GitHub Checkpoint and Exit Check

Submission Evidence Checklist

- Lab02 contains Student.java
- Lab02 contains Task1Demo.java
- Lab02 contains Task2Demo.java
- Lab02 contains OverloadDemo.java
- Lab02 contains PassByValueDemo.java
- Lab02 contains AIExperiment.java
- Lab02 contains AIExperimentDemo.java
- Lab02 contains predictions.txt
- Lab02 contains compiler_notes.txt (invalid overload error captured)
- Commits made: "CSC241 Lab02 tasks 1-4" and "Complete Lab02 integrated AIExperiment task", then pushed

Five-Minute Exit Check

1. What is the difference between a class and an object?

Answer: A class is the template/definition for a type of object; an object is an actual instance created from that class, with its own memory and field values.

2. Why does changing s1.completedCredits not change s2.completedCredits?

Answer: s1 and s2 are separate objects with separate memory for their instance variables, so a change to one object's field cannot affect the other object's field.

3. What is the method signature of enroll(String, int)?

Answer: enroll(String, int).

4. Why can two methods not be overloaded by return type alone?

Answer: The compiler resolves overloads by matching the number and types of arguments in a call; return type is not part of the method signature, so two methods differing only in return type look identical to the compiler.

5. Why can a method mutate an object through a copied reference value but cannot reassign the caller's reference variable?

Answer: The copied reference still points to the same object, so field changes through it are visible everywhere; but reassigning the local copy inside the method only changes what that local copy points to, leaving the caller's original variable untouched.